MAT-MAP

A Fare, Traffic and Crowd-Aware Companion App for Nairobi's Matatu Public Transport Network

Project Documentation

Prepared by: Group Five

Team members: Britton Kinuthia - SCT211-0225/2024
Fabiola Mukanzi - SCT211-0013/2024
Ashley Miser - SCT211-0020/2024
Micah Ade - SCT211-0014/2024

Repository: https://github.com/Mat-Map/Mat-Map

Version: 1.0

Team size: 4 members

Contents

Introduction and Background

Nairobi is one of Sub-Saharan Africa's fastest-growing cities, with a population exceeding five million and a metropolitan economy heavily dependent on daily commuter mobility. Matatus and nganyas, privately owned minibuses operating on semi-fixed routes, form the backbone of this mobility, carrying an estimated four million passengers per day across a network of routes that reaches nearly every corner of the city.

Despite the centrality of matatus to urban life, the system operates in an information vacuum. Fares are negotiated informally and fluctuate without notice. Departure times are unpredictable. Route changes happen without any public communication. Stage (bus stop) crowding is visible only to those physically present. The consequence is rational but costly: riders hedge against uncertainty by arriving early, waiting long, or switching to higher-emission alternatives such as motorcycle taxis (boda bodas) when the cost of a wasted trip becomes too high.

Existing digital tools, such as Google Maps, provide routing for matatus only in aggregate, without live fare data, crowd levels, or disruption alerts. Community-built overlays such as Ma3Route have partially addressed the information gap through social reporting, but lack structured data, predictive capability, and integration with a navigable interface.

Mat-Map addresses this gap by layering matatu-specific, community-sourced intelligence on top of Google Maps' existing transit data, rather than attempting to rebuild routing and mapping infrastructure from scratch. This narrows the engineering effort to the layer that is genuinely missing from the market — fare estimates, traffic-compensated ETAs, live stage crowd levels, and vehicle vibe — while keeping the underlying routing accuracy of an established provider. The result is a Progressive Web App (PWA), delivered through a Human-Computer Interface designed for mid-range Android browsers, that transforms crowdsourced rider reports into actionable, trip-level intelligence.

Problem Statement

Matatus and nganyas are the backbone of daily transport for the vast majority of Nairobi residents, yet the experience of planning and taking a trip remains largely undigitised and unpredictable. Google Maps' public transit layer can tell a commuter which route and stages to use, but it stops short of the information that actually determines whether a trip is a good one:

- What the trip will realistically cost — matatu fares fluctuate by time of day, weather, and route demand, and are not represented in any public API.
- Whether current road traffic will meaningfully change the journey time beyond the scheduled estimate.
- How crowded the boarding stage is right now, before the commuter even leaves the house.
- What kind of ride to expect — a loud, music-heavy "nganya" experience versus a quieter,

more subdued vehicle.

Mat-Map addresses this gap by layering matatu-specific, community-sourced intelligence on top of Google Maps' existing transit data, rather than attempting to rebuild routing and mapping infrastructure from scratch. This narrows the engineering effort to the layer that is genuinely missing from the market, while keeping the underlying routing accuracy of an established provider.

2.1 Evidence Base

The problem statement is grounded in a 116-respondent user survey conducted prior to development, which directly shaped the V1 feature prioritisation:

- **Fare estimates:** 87 of 116 respondents (75%) rated fare estimation "Very Useful" — the single highest-rated feature.
- **Traffic/delay alerts:** 78 of 116 respondents (67%) rated route-change/delay alerts "Very Useful."
- **Crowd levels:** 69 of 116 respondents (59%) rated live stage crowd levels "Very Useful."
- **Vehicle vibe (nganya vs. quiet):** cited by roughly a third of respondents as a factor influencing which vehicle they choose to board.
- **Willingness to contribute data:** 66 of 116 respondents said "Yes" and a further 33 said "Maybe" to contributing fare/crowd/vibe reports themselves — the data foundation the entire product depends on.

Project Objectives

3.1 Primary Objectives

O1. Deliver base route search on top of Google Maps' transit data, returning matatu route and stage sequences through the same interaction pattern riders already expect from Google Maps Transit.

O2. Develop a crowdsourced fare-estimation system that allows matatu riders to submit fares paid, and surfaces an aggregated, time-of-day-aware fare range to all users for a given route or leg.

O3. Implement a traffic-compensated ETA that blends Google's traffic-aware duration with any active delay or route-change reports submitted by the community, rather than relying on scheduled time alone.

O4. Surface live stage crowd levels (Low/Medium/High) so riders can gauge how busy a boarding point is before leaving the house, reducing the incentive to over-wait or abandon matatus for higher-emission alternatives.

O5. Build a vehicle "vibe" filter that lets riders indicate a preference for a loud, music-heavy "nganya" experience versus a quieter, more subdued vehicle, and ranks route options accordingly using community vibe reports.

O6. Deliver all of the above through a rider-facing Progressive Web App requiring no app-store installation, optimised for the mid-range Android browsers that make up the majority of the target user base.

Users of the System

4.1 Primary Users

Daily and semi-regular matatu commuters in Nairobi, predominantly accessing the app via mid-range Android smartphones over mobile browsers rather than dedicated app-store downloads. This user profile directly drove the decision to ship Mat-Map as a Progressive Web App (PWA) rather than a native app — removing the install-friction barrier for the target demographic.

4.2 Secondary Users

Occasional or first-time matatu riders (e.g. visitors, new residents, or commuters travelling an unfamiliar route) who rely more heavily on fare-range and vibe information to reduce the uncertainty of an unfamiliar trip, since they lack the tacit local knowledge regular commuters already have.

4.3 Data-Contributing Users

A subset of both groups above who opt in to submitting post-trip reports (fare paid, stage crowd level, vehicle vibe) through the in-app report flow. This group is functionally a distinct user type because the product's core value proposition — real, current fare/crowd/vibe data — depends entirely on their continued participation rather than on any GPS hardware or sacco-side integration.

4.4 Out-of-Scope Stakeholders (V1)

Sacco operators and matatu owners/drivers are identified as future-facing stakeholders (e.g. a sacco-side dashboard, driver-side GPS reporting) but are explicitly excluded from the V1 build. All V1 vehicle-position, crowd, and vibe signals originate from passenger-submitted reports rather than any operator or driver system, which is the key design decision that keeps a 4-week build achievable.

Literature Review

5.1 Informal Transit and the Digital Gap

Informal and semi-formal transit systems are the dominant mode of urban mobility in Kenya. Their agility (routes adjust to demand) is also their liability: the same informality that enables flexibility prevents reliable information dissemination. With the matatu network specifically, route data in Nairobi existed primarily as tacit knowledge held by drivers, conductors, and long-time commuters — undigitised, undocumented, and inaccessible to newcomers or infrequent riders.

The Digital Matatus project (2014–2015), a collaboration between the University of Nairobi and MIT, produced the first complete GTFS (General Transit Feed Specification) dataset for the Nairobi matatu network, enabling its appearance in Google Maps. However, GTFS is a static format; it captures scheduled routes, not live conditions — the gap Mat-Map is designed to close.

5.2 Crowdsourcing for Urban Transit Intelligence

The use of crowdsourcing to fill transit information gaps has been explored in both developed and developing city contexts.

5.2.1 Ma3Route (Nairobi, 2012–present)

Ma3Route is the most direct predecessor to Mat-Map in the Nairobi context. Launched in 2012, it enables riders to report traffic conditions, matatu availability, and fare levels via Twitter, SMS, and a mobile app. At its peak, Ma3Route aggregated thousands of daily reports and influenced rider decision-making across the city.

Its limitations are instructive for Mat-Map's design. Reports are unstructured (free text via social media), making automated aggregation unreliable. The service has no predictive layer, since it only reflects current conditions. Route-specific attribution is inconsistent. Mat-Map addresses these gaps through structured report schemas (fare, crowd, and vibe reports tagged to a specific stage and route) feeding a purpose-built enrichment layer on top of Google's routing data.

5.2.2 Waze and Citizen-Driven Traffic Intelligence

Waze demonstrated at global scale that passive and active crowdsourcing could produce traffic intelligence superior to sensor-based systems in cost and coverage. Key lessons applicable to Mat-Map include the importance of social and competitive incentives in sustaining contributor engagement, and the value of normalising individual reports against the aggregate to filter outliers — the same principle underlying the outlier-rejection logic in Mat-Map's report-ingestion endpoints.

5.2.3 Ushahidi and Structured Crisis Reporting

Ushahidi, also Kenyan in origin, pioneered the use of mobile-first crowdsourced reporting for high-stakes civic contexts. Its core architectural insight — that report value increases with geographic and temporal attribution — directly informs Mat-Map's data model, in which every fare and vibe report is tagged with a stage or route identifier and a timestamp.

5.3 AI and Machine Learning in Transit Prediction

Applying statistical and machine-learning methods to transit demand and pricing prediction is well studied in formal transport contexts, where models trained on historical trip data have been used to estimate short-horizon demand and pricing patterns from time, location, and weather features. Mat-Map's fare estimator follows a comparable approach at a smaller scale, starting from a statistical baseline (median fare per route and time-of-day bucket) and evolving toward a weighted model as more rider-submitted reports accumulate.

For contexts where training data is initially sparse — as Mat-Map will face in its first weeks — cold-start heuristics using time-of-day and day-of-week priors offer a practical interim solution. This is the approach taken across Mat-Map's community-driven data layer: the survey dataset seeds initial fare baselines, and the system's rules-based logic (rather than a trained model) governs traffic-ETA blending and vibe classification until enough live reports accumulate to justify a more sophisticated model.

5.4 Transit Apps in the Global South

Several transit apps have targeted comparable markets with varying degrees of success:

Platform	Market	Approach	Limitation
Ma3Route	Nairobi, Kenya	Social-media crowd reports, traffic alerts	Unstructured data, no prediction layer
Moovit	Global (50+ cities)	GTFS + live data, step-by-step navigation	Requires formal route data; poor informal transit coverage
WhereIsMyTransport	Multiple African cities	Digitising informal routes via field mapping	Supply-side focus; no live crowd/fare data
Swvl	Cairo, Nairobi	Fixed-route bus booking, app-based	Requires advance booking; not demand-responsive
Tembea (Google)	Nairobi	Matatu routing in Google Maps via GTFS	Static schedule data only; no live conditions
Transit App	North America / Europe	Real-time GTFS-RT, crowdsourced alerts	No coverage for informal/paratransit networks

Mat-Map occupies a gap none of the above fully addresses: a structured, community-driven layer specifically designed for the operational realities of Nairobi's informal matatu system, built on top of — rather than in competition with — an established routing provider.

5.5 Climate and Modal Shift in Urban Mobility

The climate rationale for improved informal transit information is grounded in modal shift theory. Uncertainty is a significant deterrent to public transport use. When travellers cannot reliably estimate journey time or cost, they systematically prefer higher-control, often higher-emission, alternatives. Nairobi-specific analysis has found that boda bodas and informal taxis, both higher per-passenger-km emitters than matatus, are disproportionately chosen when matatu availability is uncertain.

Reducing the information-uncertainty cost of matatu travel is therefore a behavioural nudge toward lower-emission modal choices, without requiring any infrastructure investment or regulatory change.

System Development Methodology

Mat-Map was built using the **Incremental (Iterative) Development Model**, in which the system is broken into a series of small, functionally complete builds ("increments"), each of which is designed, built, integrated, and demonstrated before the next increment begins. Unlike the Waterfall model, later-stage discoveries in one increment can inform the design of the next without requiring the whole project to restart from the requirements phase. Unlike a pure Scrum/Agile process, each increment here was scoped in advance around a specific, demonstrable slice of end-to-end functionality rather than a backlog re-prioritised sprint by sprint — a deliberate choice given the fixed 4-week timeline and the need for a working demo at the end of every week.

The original product concept spanned an 8-week scope that included a driver-facing Android GPS application and a USSD/SMS channel for feature-phone users. This was deliberately narrowed to the 4-week V1 scope described in this document, dropping the driver-side app and USSD channel in favour of layering intelligence on top of Google Maps' existing transit data — a scope reduction made directly in response to the timeline constraint, and itself an example of the incremental model's core principle: ship a smaller, working system first, then expand.

6.1 Increment Breakdown

Each of the four weekly increments below added a fully working, independently demonstrable capability on top of the previous increment's output, following the classic incremental-model shape of Requirements → Design → Build → Integrate → Demo, repeated per increment:

Increment	Stage Focus	System Analysis / Design Activities	End-of-Increment Demonstrable
Increment 1 (Week 1)	Foundations & Contracts	Requirements confirmed from survey data; API contract authored (`docs/api-spec.md`) before parallel build began; Postgres schema drafted; repo/CI scaffolding designed	Live search returns a real Google-Maps-based matatu route in-app (no fare/crowd/vibe yet)
Increment 2 (Week 2)	Core Enrichment	Data model implemented (`fare_reports`, `vibe_reports` tables); report-ingestion validation rules designed; seed-data ETL from survey CSV	Full route search returns fare estimate + crowd level per stage, backed by seeded real data
Increment 3 (Week 3)	Traffic ETA & Vibe Filtering	Rules-engine design for traffic-delay blending; vibe classification approach finalised (majority-vote vs. stretch classifier)	All 5 core features working end-to-end on a shared staging environment
Increment 4 (Week 4)	Field Validation & Hardening	Real-user acceptance testing (10–15 users); defect triage from field data; model recalibration	Live product demo plus a written summary of real user-report findings

System Architecture & Technology Stack

Mat-Map is built as a monorepo with three independently deployable services orchestrated by a thin backend API. The backend never invents live vehicle GPS data — it (a) calls Google Maps for the route skeleton, (b) enriches each leg with fare, crowd, vibe, and traffic-adjusted-ETA data pulled from the application database and ML service, and (c) accepts community reports that feed that database.

7.1 Component Breakdown

Layer	Specific Technology	Responsibility
Human-Computer Interface (UI/UX)	Next.js Progressive Web App (PWA), React Query for server-state caching	Search screen with Google Places autocomplete; results screen (route cards, fare range, crowd badge, traffic-adjusted ETA); embedded Google Map view with crowd-coloured stage pins; sub-3-tap post-trip report modal
Business Logic / REST API Layer	Node.js + Express	Orchestration endpoints (`/route`, `/fare`, `/eta`), report-ingestion endpoints with rate-limiting and outlier rejection, internal client for the ML microservice
Data Access / Persistence Layer	PostgreSQL with Prisma ORM	Schema and migrations for stages, routes, `fare_reports`, `vibe_reports`, and a lightweight device-based users table (no full auth required for V1)
Machine Learning / Predictive Analytics Service	Python + FastAPI (separate microservice)	Fare estimator (statistical/rule-based, evolving to weighted regression), traffic-delay rules engine, vibe classifier (majority-vote aggregation with a stretch light classifier)
External Mapping Provider	Google Maps Platform — Directions/Routes API (TRANSIT mode), traffic-aware Directions API, Places API	Route/stage skeleton and traffic-aware base ETA that Mat-Map's own services enrich
Infrastructure / DevOps	Docker Compose (local), GitHub Actions (CI/CD), Render/Fly.io (backend + ML), Vercel/Netlify (frontend)	Local multi-service orchestration, path-filtered automated testing per service, staged and production deployment, secrets and API-quota management

7.2 Data Model (Core Tables)

Table	Key Fields	Purpose
stages	id, name, lat, lng, common_routes[]	Physical matatu boarding/alighting points
routes	id, name, sacco_optional, path_stage_ids[]	A named route as a sequence of stages
fare_reports	route_id, stage_from, stage_to, fare_amount, reported_at, time_of_day, weather_flag	Crowdsourced fare data feeding the estimator
vibe_reports	route_id, vehicle_tag_optional, vibe (nganya/quiet), reported_at	Crowdsourced vehicle-vibe tags
users	id, device_id, trust_score	Lightweight, device-based identity for rate-limiting — no account/auth required in V1

*Seed strategy: **fare_reports** and **vibe_reports** are bootstrapped from the 116-response survey CSV so the application is not cold-starting with zero data on first demo, with the ML service later blending in live community reports as they accumulate.*

Team Members and Individual Contributions

The team consists of four members, each owning a distinct architectural layer described in Section 7. Responsibilities below reflect the scope each member owns per the project's role breakdown; where a specific deliverable has been independently verified against the repository, it is noted explicitly.

8.1 Fabiola Mukanzi - Human-Computer Interface (UI/UX) Engineer

Owns everything the end user directly sees and interacts with, within `apps/frontend`.

- Search screen: origin/destination input with Google Places autocomplete, and the Nganya/Quiet preference toggle.
- Results screen: route cards displaying stage-to-stage path, fare estimate range, traffic-adjusted ETA, and per-stage crowd badge.
- Map view: embedded Google Map rendering the route polyline with stage pins colour-coded by crowd level.

- Report flow: the lightweight post-trip report modal (fare paid, stage busyness, vehicle vibe) that functions as the product's core data-collection mechanism, deliberately scoped to under three taps.
- Client-side state/caching via React Query, and mobile-first responsive layout given the target user base is predominantly on phone browsers, not desktop.

8.2 Britton Kinuthia - Server-Side Application & Data Layer Engineer

Owns the orchestration API and the persistence layer, within `apps/backend`.

- `GET /route` — calls the Google Routes API and returns a normalised route + stage list.
- `GET /fare` and `GET /eta` — merge Google's raw routing/traffic data with the ML service's fare and delay outputs.
- `POST /reports/fare`, `/reports/crowd`, `/reports/vibe` — community report ingestion with device-based rate-limiting and statistical outlier rejection (e.g. rejecting a reported fare more than 5× the route median).
- PostgreSQL schema design and migrations via Prisma ORM.
- Authorship of the API contract (`docs/api-spec.md`) in the first increment, so the UI/UX and ML layers could build in parallel against a fixed interface rather than waiting on the backend sequentially.
- Internal service client (`services/mlClient.js`) for calling the ML microservice over REST.

8.3 Ashley Miser - Machine Learning / Predictive Analytics Engineer

Owns everything that turns raw community reports into predictions, within `apps/ml-service`.

- Exploratory analysis of the 116-response survey dataset in Increment 1 to calibrate initial fare and crowd baselines before any live data existed.
- Fare estimator: a statistical/rule-based model (median fare per route and time-bucket) with a planned evolution toward weighted regression incorporating time-of-day, rain, and report recency.
- Traffic-ETA blending logic: a rules engine (not a trained model) that adjusts Google's traffic-aware duration using active delay/route-change reports from the community table.
- Vibe classifier: majority-vote aggregation of `vibe_reports` per route as the V1 baseline, with a stretch goal of a lightweight text classifier over vehicle-tag descriptions if the report data proves rich enough.
- Exposing all of the above as FastAPI endpoints consumed by the backend service.

8.4 Micah Ade - DevOps / Infrastructure & CI-CD Engineer

Owns infrastructure, environments, CI/CD, and secrets management, and serves as the integration point pulling the other three services together.

- Repository initialisation and linkage of the local development environment to the remote GitHub repository (SSH-based authentication), including resolution of divergent-history conflicts arising from repository initialisation choices.
- Version-control governance: branch-protection rules on `main` (mandatory pull-request review before merge), a Conventional Commits standard (`feat:`/`fix:`/`docs:`/`chore:`/`refactor:`/`test:`) documented in `CONTRIBUTING.md`, and Issue/Pull-Request templates under `.github/` to standardise submissions.
- Monorepo directory scaffolding matching the architecture in Section 7, generated via a reusable shell script so the structure could be reliably reproduced.
- CI/CD pipeline implementation: three independent, path-filtered GitHub Actions workflows (`ci-backend.yml`, `ci-frontend.yml`, `ci-ml.yml`), each triggered only by changes within its own service directory so that, for example, a frontend-only pull request does not wait on machine-learning tests.
- Diagnosis and remediation of a CI outage in which all prior workflow runs were failing immediately due to empty workflow-definition files, and cross-triggering caused by an initially missing path-filter configuration.
- Planning of environment and secrets handling (`.env.example`, gitignored local secrets, GitHub Actions secrets for CI/CD) and of the Google Maps Platform API key provisioning and billing-cap strategy to prevent uncontrolled API spend during testing.
- Local multi-service orchestration design via Docker Compose, and staged/production deployment planning (Render/Fly.io for backend and ML service, Vercel/Netlify for the frontend).

Risks and Mitigations

Risk	Mitigation
Google Maps API quota/cost during testing	Hard billing cap and alerts configured in Increment 1; aggressive caching of route responses between fixed stage pairs, which do not change often
Cold start — no crowd/fare/vibe data at launch	Database seeded from the 116-response survey CSV in Increment 1 so every common route has a baseline before any live report arrives

Risk	Mitigation
Community reports are sparse or low-trust	Per-device rate limiting, statistical outlier rejection, and explicit "based on N reports" labelling so users can calibrate their own trust in an estimate
4-week timeline across 5 features and 4 people	API contract locked in Increment 1 so all three build layers work in parallel rather than sequentially; vibe classifier has an explicit majority-vote fallback if the stretch classifier is not ready in time
Traffic-ETA blending logic becomes disproportionately ML-heavy	Deliberately framed and implemented as a rules engine rather than a trained model, since no labelled delay dataset exists yet

Conclusion

Mat-Map addresses a well-documented and consequential gap in Nairobi's urban mobility infrastructure: the absence of reliable, structured, real-time information in the matatu system. Its V1 scope is a deliberately narrowed, four-increment build that layers matatu-specific fare, traffic, crowd, and vibe intelligence on top of Google Maps' existing transit data, rather than re-building routing infrastructure from scratch.

The incremental development model allowed the team to reduce an original 8-week concept down to an achievable 4-week build while preserving a working, demonstrable product at the end of every increment. Each of the four specialised engineering roles — Human-Computer Interface, Server-Side Application & Data, Machine Learning, and DevOps/Infrastructure — owns a distinct architectural layer, with the API contract authored in the first increment acting as the coordination point that allowed all three build layers to progress in parallel.

Grounded in a 116-respondent user survey and informed by prior work in informal-transit digitisation and crowdsourced urban intelligence, Mat-Map is designed with a clear path from a four-week prototype to a product that meaningfully reduces the information uncertainty riders face every day.